

ROS2 Based Autonomous Robo with Nvidia Jetson Orin

Nvidia Jetson Orin Nano Super



Specifications

AI Performance	67 INT8 TOPS
GPU	NVIDIA Ampere architecture with 1024 CUDA cores and 32 tensor cores
CPU	6-core Arm® Cortex®-A78AE v8.2 64-bit CPU 1.5MB L2 + 4MB L3
Memory	8GB 128-bit LPDDR5 102 GB/s
Storage	512GB NVMe
Power	7W–25W

The NVIDIA Jetson Orin Nano Super Developer Kit is a compact, yet powerful computer that redefines generative AI for small edge devices. It delivers up to 67 TOPS of AI performance—a 1.7X improvement over its predecessor—to seamlessly run a wide variety of generative AI models, like vision transformers, large language models, vision-language models, and more. It provides developers, students, and makers with the most affordable and accessible platform with the support of the NVIDIA AI software and a broad AI software ecosystem to democratize generative AI at the edge.

Robot Operating System (ROS)



The Robot Operating System (ROS) is an open-source robotics middleware framework that provides a collection of software libraries and tools for developing robotic applications. Although ROS is called an “operating system,” it is not a traditional operating system. Instead, it acts as a software framework that simplifies the development, integration, and management of complex robotic systems.

ROS provides essential functionalities such as hardware abstraction, low-level device control, package management, inter-process communication, and implementation of commonly used robotic algorithms. It follows a modular architecture in which individual processes, called *nodes*, communicate with each other through message passing. These nodes can perform tasks such as sensor data acquisition, motion control, perception, planning, mapping, and actuator control.

One of the major strengths of ROS is its extensive ecosystem of reusable software packages. These include hardware drivers, robot models, simulation environments, path-planning algorithms, perception systems, and Simultaneous Localization and Mapping (SLAM) solutions. ROS also includes powerful development and debugging tools, making it widely used in robotics research and prototyping. ROS is mainly designed for Unix-like operating systems, especially Ubuntu Linux, which is officially supported. While ROS provides flexibility and modularity, the original ROS architecture has limitations in areas such as real-time communication, scalability, reliability, and security.

Robot Operating System 2 (ROS 2)



Robot Operating System 2 (ROS 2) is the next-generation version of ROS, developed to overcome the limitations of the original ROS framework. ROS 2 introduces a redesigned communication architecture that improves real-time performance, reliability, scalability, and security for modern robotic applications. Unlike ROS, which uses a centralized communication system, ROS 2 is built on top of the Data Distribution Service (DDS), an industry-standard middleware protocol that enables fast, reliable, and distributed communication between robotic components. This makes ROS 2 more suitable for industrial automation, autonomous systems, and mission-critical robotics applications. ROS 2 retains the modular node-based architecture of ROS while introducing several advanced features such as:

- Real-time communication support
- Improved multi-robot communication
- Better security through encryption and authentication
- Cross-platform support for Linux, Windows, and macOS
- Support for embedded systems and microcontrollers

- Improved reliability and scalability
- Quality of Service (QoS) control for communication

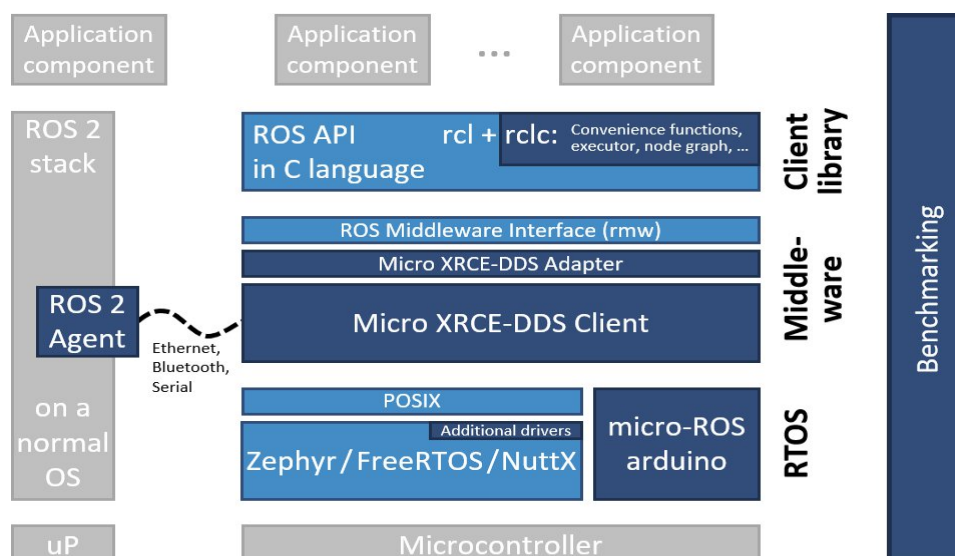
It also integrates with modern simulation and visualization tools such as Gazebo and RViz. ROS 2 is designed to work efficiently with AI systems, edge devices, RTOS-based hardware, and Industrial IoT applications.

Today, ROS 2 is becoming the preferred platform for modern robotics development because of its improved architecture, long-term support, and compatibility with advanced robotic and AI technologies.

micro-ROS on ESP32



micro-ROS is a lightweight version of ROS 2 designed specifically for microcontrollers and resource-constrained embedded systems. It extends the capabilities of ROS 2 to low-power devices such as ESP32, STM32, RP2040, and other microcontroller platforms, enabling them to participate directly in a ROS 2 robotic network.



Unlike a standard ROS 2 system that typically runs on high-performance computers like PCs or Raspberry Pi boards, micro-ROS allows small embedded controllers to function as intelligent ROS 2 nodes. This makes it possible to perform real-time hardware interfacing, sensor acquisition, actuator control, and low-level robotic operations directly on microcontrollers.

In this system, the ESP32 acts as the main embedded controller running micro-ROS firmware. It interfaces with multiple hardware peripherals including motor drivers, sensors, and battery monitoring circuits while communicating with the main ROS 2 ecosystem through USB serial.

The ESP32 is connected to a BTS7960 high-current motor driver module for controlling DC motors. Using PWM signals generated by the ESP32, the system can control motor direction, speed, and braking functions efficiently. The motor commands received from the ROS 2 network are processed by the ESP32 and translated into real-time motor control signals for robotic movement.

An MPU6050 inertial measurement unit (IMU) is connected through the I2C interface to provide acceleration and gyroscope data. This sensor enables orientation tracking, motion detection, tilt estimation, and stabilization functionalities for the robot. The ESP32 continuously reads IMU data and publishes it as ROS 2 topics through micro-ROS.

The system also includes battery voltage monitoring and State of Charge (SoC) estimation. The ESP32 reads the battery voltage using its internal Analog-to-Digital Converter (ADC) through a voltage divider circuit. Based on the measured voltage and battery characteristics, the controller estimates the remaining battery percentage and publishes the information to ROS 2 for power management and monitoring. Additionally, two ultrasonic sensors are integrated for obstacle detection and distance measurement. These sensors help the robot detect nearby objects and avoid collisions during navigation. The ESP32 processes the distance measurements and sends the sensor data to the ROS 2 network in real time.

micro-ROS uses the Micro XRCE-DDS middleware to communicate with a micro-ROS Agent running on a more powerful system such as a Raspberry Pi, Jetson, or PC. The agent acts as a bridge between the embedded microcontroller and the full

ROS 2 environment, allowing seamless communication between microcontroller nodes and higher-level robotic applications.

The complete system architecture enables distributed robotics development where:

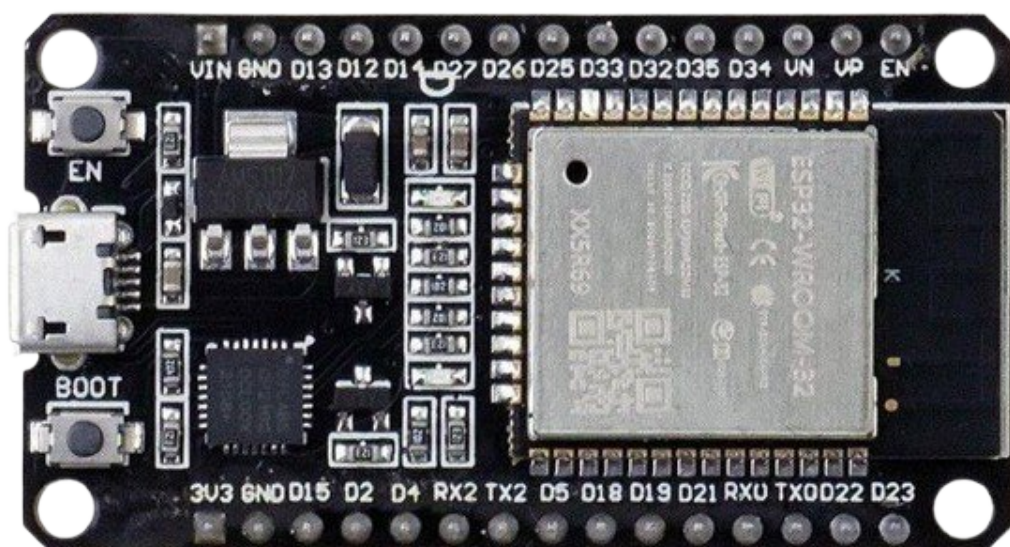
- ESP32 handles low-level real-time hardware control
- micro-ROS manages ROS 2 communication on the microcontroller
- ROS 2 on a Jetson performs advanced processing such as SLAM, navigation, AI inference, and visualization

Key advantages of using micro-ROS with ESP32 include:

- Lightweight ROS 2 integration for embedded systems
- Real-time sensor and actuator interfacing
- Communication through USB Serial or WiFi
- Low power consumption
- Scalable robotic architecture
- Easy integration with ROS 2 tools and packages
- Suitable for mobile robots, autonomous systems, and IoT robotics

This architecture is widely used in modern robotics applications where embedded controllers perform hardware-level tasks while higher-level computers handle computationally intensive operations such as autonomous navigation, machine learning, and decision-making.

ESP32 Low Level Controller



Features of ESP32 Development Board:

1. Dual-core 32-bit LX6 microprocessor
2. Integrated 2.4GHz Wi-Fi and Bluetooth (Classic + BLE)
3. 30-pin layout for compact form factor
4. 520KB SRAM + 4MB Flash memory
5. Multiple GPIOs with support for ADC, DAC, PWM, I2C, SPI, UART
6. PCB antenna for wireless communication
7. Built-in USB to Serial (CP2102/CH340) for easy programming
8. Low-power consumption with multiple sleep modes
9. Ideal for IoT, automation, robotics, and sensor networks

Specifications of ESP32 Development Board:

- Chipset: ESP-WROOM-32
- Microcontroller: Tensilica Xtensa Dual-Core 32-bit LX6
- Clock Speed: Up to 240MHz
- Flash Memory: 4MB
- SRAM: 520KB
- Wi-Fi: IEEE 802.11 b/g/n
- Bluetooth: v4.2 (Classic + BLE)
- GPIO Pins: 30
- USB Interface: Micro USB (via CP2102/CH340)
- Operating Voltage: 3.3V (logic), USB 5V input
- Mounting: Breadboard-compatible

Features

- **X2 LiDAR Sensor**

Provides real-time 360° distance scanning and obstacle detection for autonomous navigation, SLAM mapping, localization, and environment awareness in mobile robotic applications.

- **30Ah LiFePO4 Battery Pack**

High-capacity Lithium Iron Phosphate battery offering long operational backup, stable voltage output, enhanced thermal safety, deep-cycle durability, and reliable power delivery for robotic systems.

- **BTS Motor Driver Modules ×2**

High-current H-bridge motor driver modules capable of driving powerful DC motors with bidirectional control, PWM speed regulation, overcurrent protection, and efficient heat handling.

- **USB Camera Module**

Enables real-time image and video acquisition for computer vision, object detection, AI-based inspection, surveillance, and autonomous robotic perception tasks.

- **Rugged Rubber Wheels – 130 mm**

Heavy-duty rubber wheels designed for strong traction, shock absorption, terrain adaptability, and smooth movement across rough indoor and outdoor surfaces.

- **High Torque DC Motors**

Geared motors delivering high torque at lower RPM, suitable for carrying heavy payloads, climbing uneven terrain, and ensuring stable robotic movement under load conditions.

You can implement:

- **Full SLAM Mapping and Autonomous Navigation**

2D SLAM

- Cartographer
- GMapping
- SLAM Toolbox

Features:

- live map creation
- occupancy grid mapping
- autonomous exploration
- localization
- map saving/loading
- create maps
- remember environments
- navigate autonomously
- avoid obstacles
- move to target coordinates

This alone is a full industrial AMR foundation.

- **ROS2 Navigation Stack (Nav2)**

With Nav2 you get:

- path planning
- obstacle avoidance
- waypoint following
- autonomous docking
- recovery behaviours
- dynamic replanning

- **Visual AI with USB Camera**

The Orin Nano Super is extremely good for AI vision.

Object Detection

- YOLOv8
- TensorRT optimized inference

Features

- person detection
- vehicle detection
- object tracking
- safety monitoring
- warehouse navigation

Sensor Fusion

Very important in robotics.

You can fuse:

- MPU6050
- wheel odometry
- LiDAR odometry
- visual odometry

using:

- robot_localization
- EKF
- UKF

This gives:

- stable pose estimation
- better localization
- reduced drift
- smoother navigation

- **Autonomous Following Robot**

Features:

- human following
- target following
- dynamic tracking
- distance maintenance

Algorithms:

- DeepSORT
- ByteTrack
- KCF/CSRT

- **ROS2 Topics You Can Learn Properly using this**

- Nodes
- Topics
- Services
- Actions
- TF2
- URDF
- RViz2
- Gazebo
- rosbag2
- lifecycle nodes
- DDS middleware
- Nav2
- MoveIt basics
- robot_localization
- sensor fusion
- micro-ROS

- **ROS2 + AI Combination**

You can run:

- ROS2
- CUDA
- TensorRT
- PyTorch
- OpenCV

You can make:

- autonomous patrol robot
- warehouse AMR
- AI surveillance robot
- face recognition robot
- smart delivery robot
- industrial inspection robot

- **Digital Twin Simulation**

You can create:

- URDF robot model
- Gazebo simulation
- Isaac Sim integration

Recommended Architecture for Jetson Orin Nano Super Dev

- Ubuntu 22.04
- ROS2 Humble
- Nav2
- SLAM
- AI inference
- vision processing